SQL Subqueries & CTEs

A beginner's guide with simple, runnable examples

What you will learn

- What a **subquery** is and the 4 common types (scalar, multi-row, correlated, EXISTS)
- What a **CTE** (Common Table Expression) is and how `WITH` works
- How to use multiple CTEs and a first look at recursive CTEs
- When to choose a subquery vs a CTE, plus common mistakes
- Practice questions with answers

All examples use standard SQL and work in SQL Server, PostgreSQL, MySQL 8+, Oracle and Microsoft Fabric / Synapse (minor syntax notes are marked).

1. Sample Data Used in This Guide

Every example below uses these two small tables. Create them once and try each query yourself.

```
CREATE TABLE departments (
    dept_id    INT PRIMARY KEY,
    dept_name  VARCHAR(30),
    location   VARCHAR(30)
);

CREATE TABLE employees (
    emp_id   INT PRIMARY KEY,
    name     VARCHAR(30),
    dept_id  INT,
    salary   INT
);

INSERT INTO departments VALUES
 (10,'Engineering','Chennai'), (20,'Sales','Bengaluru'),
 (30,'Finance','Chennai'),     (40,'Marketing','Mumbai');

INSERT INTO employees VALUES
 (1,'Asha',10,50000), (2,'Ravi',10,60000), (3,'Meena',20,45000),
 (4,'Karthik',20,70000), (5,'Divya',30,55000), (6,'Suresh',30,40000),
 (7,'Priya',10,80000);
```

employees

emp_id	name	dept_id	salary
1	Asha	10	50000
2	Ravi	10	60000
3	Meena	20	45000
4	Karthik	20	70000
5	Divya	30	55000
6	Suresh	30	40000
7	Priya	10	80000

departments

dept_id	dept_name	location
10	Engineering	Chennai
20	Sales	Bengaluru
30	Finance	Chennai
40	Marketing	Mumbai

Note: department 40 (Marketing) has no employees. We will use that later.

2. Subqueries

A **subquery** is a query written inside another query, wrapped in parentheses. The inner query runs first and hands its result to the outer query. Think of it as: *"first answer this small question, then use that answer to answer the big question."*

> **Where can a subquery go?** In `WHERE` (filter rows), in `FROM` (acts like a temporary table, called a derived table), or in `SELECT` (returns one value per row).

2.1 Scalar subquery (returns ONE value)

Goal: find employees who earn more than the company average salary.

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);   -- inner query returns 57142.86
```

Result:

name	salary
Ravi	60000
Karthik	70000
Priya	80000

The inner query returns a single number, so you can use =, >, <, >= etc. If it returns more than one row you will get an error.

2.2 Multi-row subquery with IN (returns a LIST)

Goal: list employees who work in departments located in Chennai.

```
SELECT name, dept_id
FROM employees
WHERE dept_id IN (SELECT dept_id
                  FROM departments
                  WHERE location = 'Chennai');       -- inner query returns 10, 30
```

Result:

name	dept_id
Asha	10
Ravi	10
Priya	10
Divya	30
Suresh	30

Use IN / NOT IN when the inner query returns many values. Beware: NOT IN returns no rows if the list contains a NULL. Prefer NOT EXISTS in that case.

2.3 Subquery in FROM (derived table)

Goal: show departments whose average salary is above 50,000.

```
SELECT dept_id, avg_salary
FROM (SELECT dept_id, AVG(salary) AS avg_salary
      FROM employees
      GROUP BY dept_id) AS dept_avg           -- alias is REQUIRED here
WHERE avg_salary > 50000;
```

Result:

dept_id	avg_salary
10	63333.33
20	57500.00

A subquery in FROM must have an alias (dept_avg). The outer query treats it just like a real table.

2.4 Correlated subquery (inner query depends on the outer row)

Goal: find employees who earn more than the average of THEIR OWN department.

```
SELECT e.name, e.dept_id, e.salary
FROM employees e
WHERE e.salary > (SELECT AVG(x.salary)
                  FROM employees x
                  WHERE x.dept_id = e.dept_id);   -- uses e.dept_id from the outer query
```

Result:

name	dept_id	salary
Priya	10	80000
Karthik	20	70000
Divya	30	55000

The inner query re-runs for every outer row (conceptually). That makes it powerful but can be slow on very large tables.

2.5 EXISTS / NOT EXISTS (does a matching row exist?)

Goal: find departments that have NO employees.

```
SELECT d.dept_name
FROM departments d
WHERE NOT EXISTS (SELECT 1
                  FROM employees e
                  WHERE e.dept_id = d.dept_id);
```

Result:

dept_name
Marketing

EXISTS only checks whether at least one row is found, so what you SELECT inside (here 1) does not matter. It is fast and NULL-safe.

Subquery cheat sheet

Type	Returns	Used with	Example idea
Scalar	1 value	=, >, <, >=	salary > (SELECT AVG(...))
Multi-row	A column of values	IN, NOT IN, ANY, ALL	dept_id IN (SELECT ...)
Derived table	A full table	FROM (...) AS alias	Aggregate first, filter after
Correlated	Depends on outer row	=, >, < with outer column	Compare to own-group average
EXISTS	TRUE / FALSE	WHERE [NOT] EXISTS	Rows with / without matches

3. CTEs (Common Table Expressions)

A **CTE** is a temporary, named result set that you define at the top of a query using the `WITH` keyword. You then use that name in the main query like a table. It exists only for that one statement.

Basic syntax

```
WITH cte_name AS (
    SELECT ...            -- any normal SELECT query
)
SELECT *
FROM cte_name;            -- use it like a table
```

Why use a CTE? It makes long queries readable (top-to-bottom, step by step), lets you reuse the same logic more than once in the same query, and avoids deeply nested subqueries.

3.1 Your first CTE

Goal: same as 2.3, rewritten with a CTE. Departments with average salary above 50,000.

```
WITH dept_avg AS (
    SELECT dept_id, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY dept_id
)
SELECT dept_id, avg_salary
FROM dept_avg
WHERE avg_salary > 50000;
```

Result:

dept_id	avg_salary
10	63333.33
20	57500.00

Same result as the derived table, but the logic is named (dept_avg) and sits at the top, so the final query reads like plain English.

3.2 CTE joined back to a table

Goal: rewrite 2.4. Show employees earning more than their own department average, and display that average too.

```
WITH dept_avg AS (
    SELECT dept_id, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY dept_id
)
SELECT e.name, e.salary, d.avg_salary
FROM employees e
JOIN dept_avg d ON e.dept_id = d.dept_id
WHERE e.salary > d.avg_salary;
```

Result:

name	salary	avg_salary
Karthik	70000	57500.00
Divya	55000	47500.00
Priya	80000	63333.33

A CTE usually reads and performs more clearly than a correlated subquery, and it lets us show avg_salary in the output.

3.3 Multiple CTEs (step by step)

Goal: find departments whose average salary is above the company-wide average. Separate CTEs with a comma; only one WITH is needed.

```
WITH dept_avg AS (
    SELECT dept_id, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY dept_id
),
company_avg AS (
    SELECT AVG(salary) AS avg_salary
    FROM employees
)
SELECT d.dept_name, da.avg_salary
FROM dept_avg da
JOIN departments d  ON d.dept_id = da.dept_id
CROSS JOIN company_avg ca
WHERE da.avg_salary > ca.avg_salary;       -- company average = 57142.86
```

Result:

dept_name	avg_salary
Engineering	63333.33
Sales	57500.00

A later CTE can also refer to an earlier one in the same WITH block.

3.4 A first look at recursive CTEs

A recursive CTE refers to itself. It is used for sequences and hierarchies (for example org charts or folder trees). Here is the simplest case: counting 1 to 5.

```
WITH numbers AS (
    SELECT 1 AS n                -- anchor: the starting row
    UNION ALL
    SELECT n + 1                 -- recursive part: builds on the previous row
    FROM numbers
    WHERE n < 5                  -- stop condition (avoid infinite loops!)
)
SELECT n FROM numbers;
```

Result:

n
1
2
3
4
5

Syntax note: PostgreSQL and MySQL need WITH RECURSIVE; SQL Server and Oracle use plain WITH. Always include a stop condition.

4. Subquery vs CTE: Which One Should I Use?

	Subquery	CTE
Where written	Inside the query (WHERE / FROM / SELECT)	At the top, using WITH
Readability	Gets hard to read when nested 2-3 levels	Reads top to bottom, step by step
Reuse in one query	Must be repeated each time	Define once, use many times
Recursion	Not possible	Possible (recursive CTE)
Scope	That one place only	The whole statement that follows
Best for	Quick, small one-off filters	Multi-step logic, long or shared logic

Rule of thumb: use a simple subquery for a quick one-line filter (like `salary > (SELECT AVG(salary) ...)`). Switch to a CTE when the query has several steps, needs the same logic twice, or starts to look like nested Russian dolls.

Performance note: in most modern databases, a CTE and an equivalent subquery are optimized very similarly, so choose mainly for clarity. A CTE is **not** automatically stored on disk or faster. It is just a named query for that statement. (If you need to reuse a result across several statements, look at temp tables or views.)

5. Common Mistakes

- **Scalar subquery returns more than one row.** Error like "subquery returned more than 1 value". Use IN, or add aggregation / TOP 1.
- **Missing alias on a derived table.** A subquery in FROM needs an alias in most databases (AS dept_avg).
- **NOT IN with NULLs.** If the list contains a NULL, NOT IN returns nothing. Use NOT EXISTS instead.
- **Forgetting the comma between CTEs** or repeating the word WITH for the second CTE.
- **Using a CTE after the statement ended.** A CTE lives for one statement only. Run the next query and it is gone.
- **Recursive CTE with no stop condition.** This loops forever (databases usually cap it, e.g. MAXRECURSION 100 in SQL Server).
- **Selecting more columns than needed in IN subqueries.** The inner SELECT for IN must return exactly one column.

6. Practice Questions

Try these on the sample data before looking at the answers.

- **Q1.** List employees who earn less than the company average (scalar subquery).
- **Q2.** List employees who work in the Sales department (IN with a subquery on departments).
- **Q3.** Show the highest-paid employee in each department (correlated subquery).
- **Q4.** Using a CTE, show departments that have more than 2 employees.
- **Q5.** Find departments located in Chennai that have at least one employee (EXISTS).

Answers

```sql
-- Q1  (Asha, Meena, Divya, Suresh)
SELECT name, salary FROM employees
WHERE salary < (SELECT AVG(salary) FROM employees);

-- Q2  (Meena, Karthik)
SELECT name FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments WHERE dept_name = 'Sales');

-- Q3  (Priya, Karthik, Divya)
SELECT e.name, e.dept_id, e.salary
FROM employees e
WHERE e.salary = (SELECT MAX(x.salary) FROM employees x WHERE x.dept_id = e.dept_id);

-- Q4  (Engineering)
WITH dept_count AS (
    SELECT dept_id, COUNT(*) AS emp_count
    FROM employees
    GROUP BY dept_id
)
SELECT d.dept_name, dc.emp_count
FROM dept_count dc
JOIN departments d ON d.dept_id = dc.dept_id
WHERE dc.emp_count > 2;

-- Q5  (Engineering, Finance)
SELECT d.dept_name
FROM departments d
WHERE d.location = 'Chennai'
  AND EXISTS (SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id);
```

Quick Recap

- A **subquery** is a query inside a query. It can return one value, a list, a table, or TRUE/FALSE.
- A **CTE** is a named temporary result defined with WITH, which makes multi-step queries easy to read.
- Start with simple subqueries; reach for CTEs when queries grow long or logic repeats.
- Practice on small tables, check each inner query on its own first, then combine.